Embedded Systems (M233)

Course summary — lectures, labs and the skills the midterms grade

The whole course on three pages, from the first blinking LED to the non-blocking FSM.

The board and the model of computation

- Arduino Uno, ATmega328P: 8-bit AVR, 16 MHz, 5 V logic, 14 digital pins (D0–D13, six PWM: 3, 5, 6, 9, 10, 11), six analog inputs A0–A5 (10-bit ADC, 0–1023), 32 KB flash, 2 KB SRAM, 1 KB EEPROM.
- The sketch runs `setup()` once, then `loop()` forever; nothing else — no threads, no OS; concurrency is an illusion you build with timers.
- `analogWrite(pin, v)`: 8-bit PWM, duty $v/256$; `analogRead` is 0–1023 over 0–5 V; `map()` never clamps — constrain after it.

Digital I/O — the rules that cost points

- `pinMode` before use; a pin never configured still works — as an input — because `INPUT` is the reset state.
- Buttons to GND with `INPUT_PULLUP`: pressed reads `LOW`, no external resistor, and a cut wire reads "not pressed" (the safe failure). Wiring to 5 V without a pull-down floats and is the classic wiring fault.
- Floating input: no pull resistor, open switch ⇒ the pin picks up ambient noise, reads randomly — seen on the serial plotter as flicker.
- LED: $R = (V_{cc} - V_f)/I$ — for 5 V, 2 V, 10 mA: $R = 300\,\Omega$ (use $330\,\Omega$); always round up, always in series with the LED.
- Transistor driver for loads over ∼20 mA (motor, relay): base resistor, flyback diode on inductive loads.

Time — delay vs millis

- `delay()` is blocking: nothing else runs; two `delay(500)` in the sketch means a press sampled once a second at best.
- The non-blocking pattern: `if (millis() - t0 >= interval) { t0 = millis(); ... }`; unsigned arithmetic survives the 49.7-day wrap.
- Debounce: a real contact bounces for 20–50 ms; require a *stable* raw level for 25 ms before accepting a change; restart the timer on every bounce.
- Edge detection: count on the press *edge* (level became pressed), not the level — otherwise holding the button auto-repeats.
- `Serial.parseInt()` leaves the newline in the buffer — the next `read()` returns 10; read or drain the line after it.

Finite state machines — the exam pattern

- State × event → next state × outputs; guard conditions decide rows; a move that "always finishes" is a state, not an event.
- Implementation: `enum class` for states, `switch` in `loop()`, one timestamp per timer (move end, blink), never `delay`.
- Blink while moving: an independent 500 ms toggle, not a second blocking loop.
- Audit table: list the states, list the events, fill every cell — an empty cell is a spec bug, and the midterms put one there on purpose.

Energy — the battery-node arithmetic

- Average current: $I_{avg} = \frac{I_{active}t_{active}+I_{sleep}t_{sleep}}{T_{cycle}}$; lifetime $= \frac{\text{capacity (mAh)}}{I_{avg}\ \text{(mA)}}$.
- Stretching the cycle helps only until $I_{avg} \to I_{sleep}$: the sleep floor caps the lifetime no matter how rare the sends.
- Three AA in series: 4.5 V, capacity *unchanged* (series adds voltage); parallel adds capacity at the same voltage.
- Reading a datasheet budget: CPU, radio, sensors, duty cycle — every row in mA·s, then divide by the period.

Platform choice and I/O inventory

- Choose the platform by: real-time needs (interrupts vs OS jitter), power (sleep current), I/O (ADC bits, PWM channels), cost, and toolchain — not by habit.
- I/O inventory: for each peripheral, the pins it uses, the electrical level, active state, and the protocol (UART 9600 ≈ 960 characters/s, I2C two wires with addresses, SPI four wires with chip selects).
- Two boards over Serial: define the frame (start byte, payload, checksum), block on `available()`, never assume the message arrives whole.

The five skills the midterms grade

1. **Energy budget** — compute I_{avg}, lifetime in days, name the sleep floor.
2. **Platform choice** — justify by requirement, not preference; say which requirement kills which platform.
3. **I/O inventory** — pins, levels, protocols, the conflicts (two peripherals claiming one pin).
4. **FSM audit** — full table, no empty cells, guards explicit, non-blocking sketch.
5. **Code reading** — find the blocking delay, the missing pull-up, the auto-repeat, the parseInt newline, the un-clamped `map()`.

The recurring traps

`delay` anywhere in an FSM question • level test where an edge is required • button to 5V with no pull-down
`INPUT_PULLUP` and expecting HIGH when pressed • LED without its resistor • serial reading without `available()`
`map()` used as a clamp • PWM pin assumed analog • debounce that only delays, never checks stability.